

Convolve 3.0

Final Report

Team: iitrnoobs

1] INTRODUCTION

Effective risk management is vital for achieving robust financial operations, especially within the banking sector. As financial institutions navigate an increasingly complex market landscape, embracing advanced, data-driven strategies to address potential risks is not just advantageous—it is essential.

With an array of advanced AI models and frameworks at its disposal for an efficient system of issuing credit cards, Bank A now wants to add another robust framework, but this time for its existing customers. For this the bank has decided to create a model for predicting the “**Behaviour Score**” of customers and predict the probability of its existing customers defaulting on their credit cards in the future. This model will then be used for several portfolio risk management activities.

This report provides a detailed overview of the model developed by our team, including its design, methodology, training process, and also the data analysis and preprocessing pipeline. In addition to this our report includes several key insights and observations on the behaviour and lifestyle of customers. With this model in their arsenal, Bank A will become more efficient in its risk management, strengthening its financial stability, improve customer trust, and optimize overall performance.

2] Data Analysis and Data Preparation

A) Feature Selection

I. Using Catboost

We first trained a simple CatBoostClassifier model to find important features by comparing feature importance scores of the columns.

CatBoost is a high-performance open source library for gradient boosting on decision trees

CatBoostClassifier (model for classification tasks in the library) was chosen due to its **native support** for NaN (Not an Integer) values and good performance without parameter tuning. The top 100 features were identified from the scores.

II. Using Mann - Whitney U Test

The **Mann-Whitney U test** is a non-parametric test used to assess whether two independent groups have significantly different distributions. It is often used when the assumptions of normality and homogeneity of variance are not met.

For each feature, we divided the data into two groups based on the target variable (Group 0 for target = 0 and Group 1 for target = 1) and performed the Mann-Whitney U test. The null hypothesis (H_0) assumes equal distributions between the two groups, while the alternative hypothesis (H_1) assumes different distributions.

The test generates a U-statistic value using the group sizes and ranks of the observations of the groups, and a p value assuming a normal distribution of the U-statistic value (which is a good approximation for large group sizes). The lower the p value, the higher is the difference in the distribution of the two groups.

Formula for U-statistic:

$$U_1 = n_1 n_2 + (n_1(n_1 + 1)/2) - R_1$$

$$U_2 = n_1 n_2 - U_1$$

U = lower of U_1 and U_2

Where

n_1 : Number of observations in Group 1

n_2 : Number of observations in Group 2

R_1 : The sum of the ranks of the observations in Group 1 (after combining the data from both groups and ranking all values).

We chose to use this test for feature selection because this test doesn't assume normal distributions for the data being compared, and the majority of columns didn't have normal distribution of values.

We ran this test on the full dataset as well as 9 subsets and compared the top 100 columns across all subsets and the full dataset as well the top 100 columns from CatBoostClassifier feature importance scores to select a set of 100 important columns. The results of the test on the subsets and full dataset are in Appendix A

III. Using ANOVA Test

We also ran a parallel feature selection algorithm using ANOVA test to find useful features.

Analysis of variance (ANOVA) is a statistical test that simultaneously compares means across several groups to determine if observed differences are due to chance or reflect genuine distinctions.

Formula of ANOVA test:

$$F = MST/MSE$$

Where

F = ANOVA coefficient

MST = Mean sum of squares due to treatment

MSE = Mean sum of squares due to error

In a similar way to above, we selected the top 100 important features

Finally we compared both of these sets to select a set of 30 important columns with a mix of transaction_attribute, bureau_enquiry, bureau and onus_attribute columns.

B) Pre-processing

I. Filling NAN

For the columns chosen, we analysed the distribution and range of values in the columns. We found that for all chosen columns the values range from very small (<-1) to very large positive values. So we decided to fill the missing values with a **large negative value** of -10 as a way for models to distinguish between these rows and other rows, as we didn't have enough info to properly fill the NaN values. Filling with a large negative value offsets this issue by making models consider such rows as a separate group.

II. Train_Test_Split

The dataset was then split into a training set and a validation set using Scikit Learn's `train_test_split`, with a ratio of 80% training set and 20% validation set for model training.

III. Scaling

As the range of values for various columns vary in a wide range of magnitudes, we scaled the data using Scikit Learn's **StandardScaler** to ensure a uniform range for all features (columns), as it has been shown to improve model quality by enabling the model to learn patterns without getting hindered by the huge range of values.

3] Model Training and Tuning

A) Model Selection

We decided to train, tune and combine four different types of models, namely -

- DecisionTreeClassifier from Scikit Learn
- XGBClassifier from XgBoost
- CatBoostClassifier from CatBoost
- LGBMClassifier from LightGBM

I. Decision Tree

A **Decision Tree** is a supervised learning algorithm used for both classification and regression tasks. It predicts the target variable by learning a series of simple decision rules based on the data features, forming a tree-like structure.

How It Works

1. Tree Structure:

- **Root Node:** Represents the entire dataset.
- **Internal Nodes:** Subsets of data split based on features.
- **Leaf Nodes:** Represent final predictions (class labels for classification or numerical values for regression).

2. Data Splitting:

- The algorithm selects the best feature and threshold at each node to split the data, aiming to improve the "purity" of subsets.
- **For classification:** Measures like Gini Impurity or Entropy are used to evaluate splits.
- **For regression:** Splits are chosen to minimize the variance of the target variable.

3. Stopping Criteria:

- Splitting stops when:
 - A maximum depth is reached.
 - The number of samples in a node is too small.
 - Further splits do not significantly improve the subset's purity.

4. Prediction:

- **Classification:** Follows the decision rules to reach a leaf node, where the predicted class label is assigned.
- **Regression:** Predicts the average value of the target variable in the leaf node.

Advantages:

- Simple to understand and interpret.
- Handles both numerical and categorical data.
- Requires minimal data preprocessing.

Disadvantages:

- Prone to overfitting, especially with deep trees.

- Sensitive to small changes in data, which can result in different splits.
- Often benefits from ensemble methods (e.g., Random Forests) for better accuracy and generalization

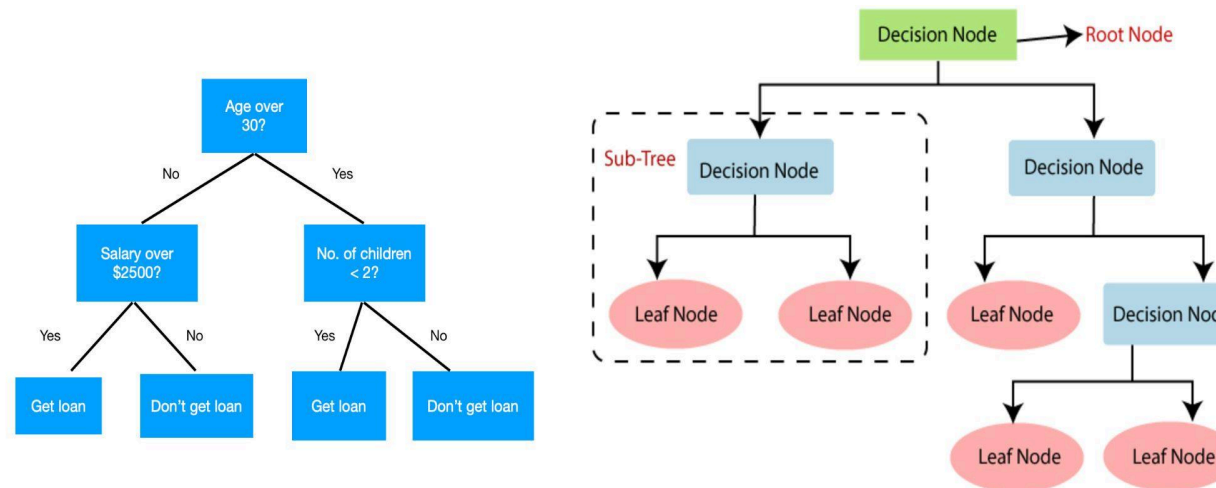


Fig: Simplified diagram of working of Decision Tree

We chose to train a Decision Tree because of its interpretability, crucial for financial institutions like banks due to the following reasons:

1. **Regulatory Compliance:** Simplifies rule adherence and audit processes.
2. **Risk Management:** Helps understand factors influencing predictions, aiding better risk control.
3. **Trust and Transparency:** Builds trust by explaining decisions and reducing legal issues.
4. **Bias Detection and Fairness:** Identifies and addresses bias, ensuring fairness.
5. **Model Monitoring and Maintenance:** Facilitates model updates as markets evolve.
6. **Customer Experience:** Enhances satisfaction by clarifying decision-making.
7. **Operational Efficiency and Accountability:** Improves workflows, collaboration, and error tracing.
8. **Ethical Considerations:** Ensures ethical decision-making with clear, socially-conscious models.

Although we were aware of more powerful models (CatBoost, XgBoost and LightGBM), since these models are less interpretable, we spent time developing a robust and well generalizable Decision Tree model. We did make the other more powerful models to complement and provide diversity to the algorithm users.

II. XGBoost

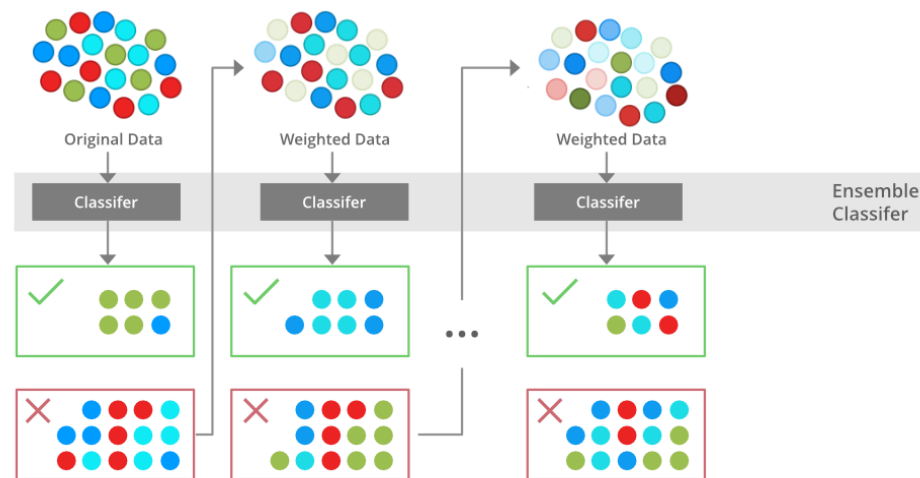
XGBoost, which stands for Extreme Gradient Boosting, is a scalable, distributed **gradient-boosted decision tree (GBDT)** machine learning library.

A Gradient Boosting Decision Trees (GBDT) is a decision tree ensemble learning algorithm for classification and regression. Ensemble learning algorithms combine multiple machine learning algorithms to obtain a better model.

The term “gradient boosting” comes from the idea of “boosting” or improving a single weak model by combining it with a number of other weak models in order to generate a collectively strong model. Gradient boosting is an extension of boosting where the process of additively generating weak models is formalized as a gradient descent algorithm over an objective function. Gradient boosting sets targeted outcomes for the next model in an effort to minimize errors. Targeted outcomes for each case are based on the gradient of the error (hence the name gradient boosting) with respect to the prediction.

With XGBoost, trees are built in parallel, instead of sequentially like GBDT. It follows a level-wise strategy, scanning across gradient values and using these partial sums to evaluate the quality of splits at every possible split in the training set.

A simple diagram illustrating how XGBoost works is as shown below -

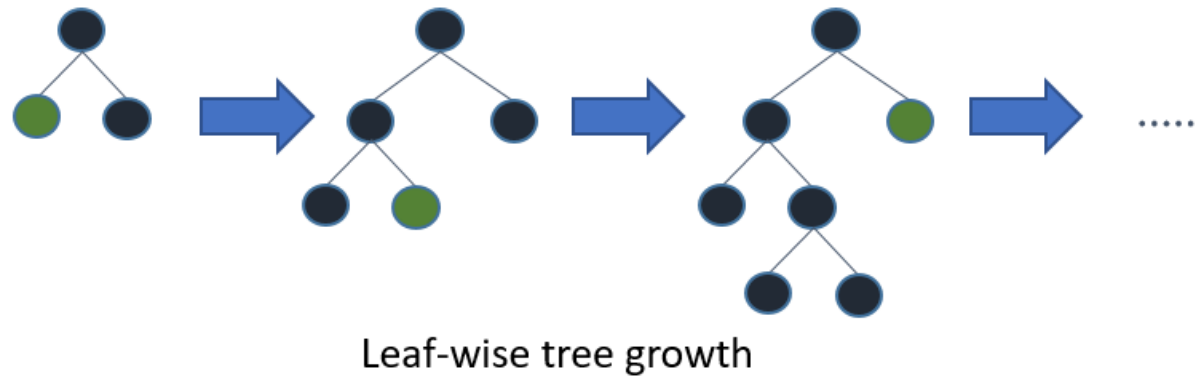


III. LightGBM

LightGBM, like XGBoost is a gradient boosting method, which constructs a strong learner by sequentially adding weak learners in a gradient descent manner. It optimizes memory usage and training time with techniques like **Gradient-based One-Side Sampling (GOSS)**. Additionally, LightGBM employs histogram-based algorithms for efficient tree construction.

Gradient Based One Side Sampling is a novel sampling method which down samples the instances on the basis of gradients. It retains instances with large gradients while performing random sampling on instances with small gradients.

The leaf growth of a simple LightGBM model is as shown below -



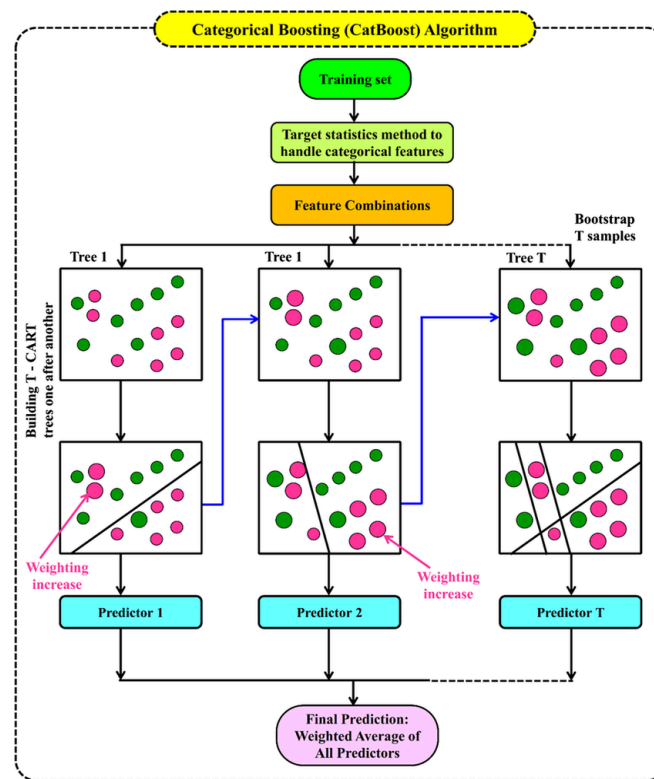
IV. CatBoost

CatBoost is another of the popular gradient boosting algorithms, developed by Yandex researchers. It is particularly efficient in **handling categorical features**, and also known to perform well **without extensive tuning**. Two of its main features are -

- **Ordered Boosting:** Traditional gradient boosting methods are prone to prediction shifts caused by target leakage, primarily when the model uses the entire dataset to determine splits. CatBoost addresses this issue with ordered boosting, a technique that creates several permutations of the data and uses only past observations for each permutation when calculating leaf values.
- **Efficient Handling of Categorical Features:** Categorical features often pose challenges for machine learning models because they cannot be directly processed like numerical data. CatBoost natively handles categorical data by

automatically determining the best way to represent these features, significantly reducing the need for manual preprocessing.

A general flow diagram of a CatBoost model is given below -



B) Training, tuning and output generation

- We used the training set generated earlier (using Scikit Learn's `train_test_split`) for training and the validation set for comparison and validation purposes.
- For hyperparameter tuning, instead of simple techniques like `GridSearchCV` or `RandomSearchCV` (from Scikit Learn), we used the **Optuna library**.
- For maximum generalization, we use **StratifiedKFold** (from Scikit Learn) over 5 folds to find the best hyperparameters.

I. Stratified K Fold

To achieve maximum generalization, we use the `StratifiedKFold` algorithm from Scikit Learn to make pairs of subsets, where one is a training set consisting of $4/5^{\text{th}}$ of the original training set and the other a validation set consisting of $1/5^{\text{th}}$ of the original training set.

What `StratifiedKFold` does instead of simple division is that it **preserves the distribution of majority and minority class** in the dataset. Since the dataset had very few instances of the minority class, dividing randomly could have generated subsets where either none or all of the minority class elements would have been present. We make 5 such different pairs.

II. Hyperparameter Optimization

We run a loop five times to generate a pair of subsets (training and validation subset) using `StratifiedKFold`, and for that pair use the `Optuna` library for optimization. This loop is run separately for each model.

`Optuna` is a hyperparameter optimization libraries which use the powerful technique of Bayesian Optimization to find best set of hyperparameters

Bayesian optimization is a powerful technique for optimizing hyperparameters in machine learning models, particularly when the search space is large, expensive, or time-consuming to evaluate. It is an efficient method that builds a probabilistic model to guide the search for the best hyperparameters.

Working of Bayesian Optimization:

1. Surrogate model:

Bayesian optimization uses a surrogate model (often a Gaussian Process, but other models can be used as well) to approximate the objective function that we want to optimize. The surrogate model helps predict the performance of different hyperparameter configurations without directly evaluating the model for each configuration, which can be computationally expensive.

2. Acquisition Function:

The acquisition function is used to decide which hyperparameters to evaluate next based on the surrogate model. It takes into account both the predicted performance and the uncertainty of the model. Common acquisition functions include:

- **Expected Improvement (EI):** Chooses points that are expected to provide the best improvement over the current best result.
- **Probability of Improvement (PI):** Chooses points that have a high probability of improving the best current result.
- **Upper Confidence Bound (UCB):** Balances exploration and exploitation by considering the uncertainty of the surrogate model.

3. Iterative Process:

The optimization process is iterative. The steps involve:

- Evaluating the surrogate model to find the hyperparameters that maximize the acquisition function.
- Testing the selected hyperparameters by evaluating the actual objective function (e.g., training the model and calculating validation accuracy).
- Updating the surrogate model with the new results.
- Repeating the process until convergence or a predefined stopping criterion (like a budget of evaluations).

4. Exploration vs. Exploitation:

Bayesian optimization tries to balance exploration (searching in regions of the hyperparameter space with high uncertainty) and exploitation (focusing on regions where the surrogate model predicts high performance). The acquisition function typically guides this balance.

For the loop we run for each model, we define an appropriate parameter space to search, which was made using our experience of working with similar datasets and some rudimentary experiments.

III. Model evaluation

The 5 sets of hyperparameters generated for each model were compared using their performance on the respective validation sets using **ROC AUC score**, or the Receiver Operating Characteristic Area Under the Curve.

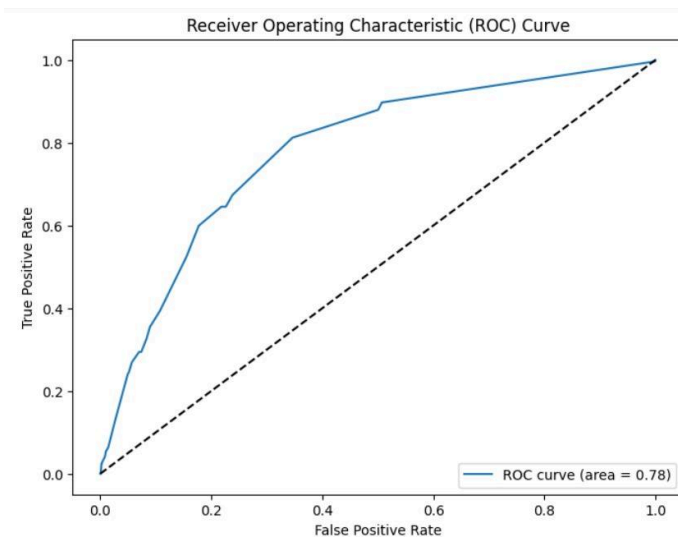
This performance metric was chosen to find the model which can best separate the majority (0 bad flag ones) and minority (1 bad flag ones) classes, and because classical metrics like precision and recall are not very good to compare models

trained in highly imbalanced datasets, as they tend to focus on the performance of models in the minority class, and models tend to perform poorly in predicting the minority class, but what is more important is the ability to properly distinguish the classes rather than exactly predicting the class.

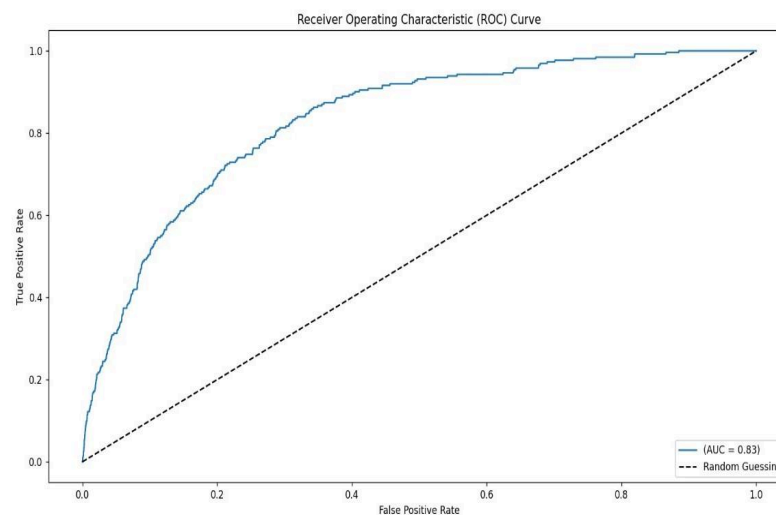
Since ROC AUC score is made using the probabilities of the model for various classes, a good ROC AUC score shows the model's ability to generate values of probability in different ranges (For example: between 0.1-0.4 for majority class and 0.5-0.9 for minority class). Also the ROC AUC score is independent of the decision threshold used for class prediction.

As a final evaluation of the models with the hyperparameters found using the above loops, we check their performances on the original validation set. Since this set was not used anytime during model training, a reasonably good performance (checked using ROC AUC score) confirmed the generalizability of the model.

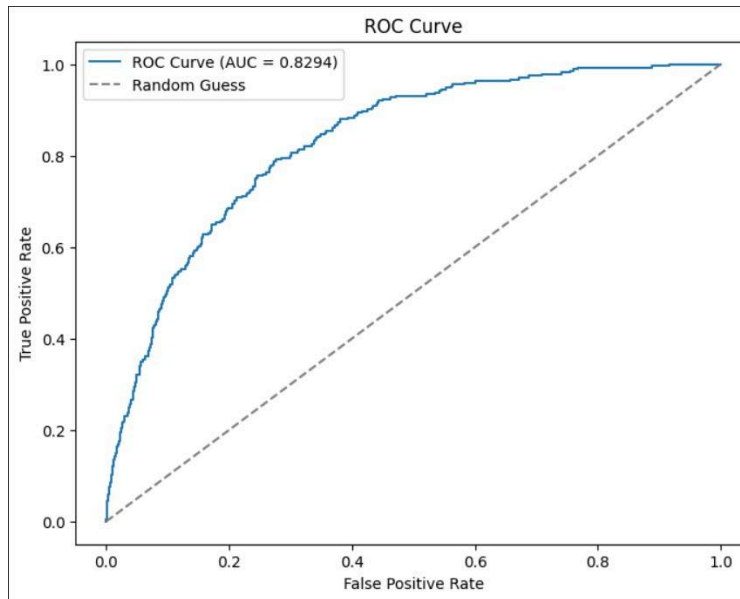
The ROC AUC curves of all of our models on the validation set generated by train_test_split are -



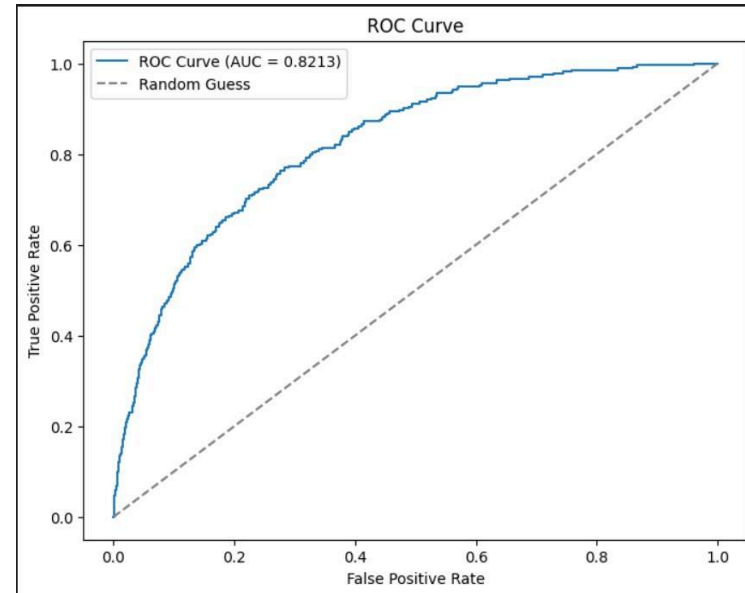
For Decision Tree model



For XgBoost model



For LightGBM model



For CatBoost model

After confirmation of all models as robust and generalizable, we proceed to the final step

IV. Combining Models

To improve the capability of our algorithm, we decided to combine the predictions (probabilities) of the four trained models. Since there is a good chance that different models learnt different patterns in the dataset, combining them can combine the power of all of the models to provide better predictions.

For combining the models, we first generated the probabilities on the test set (validation_set_to_be_shared.csv) and took their **average**. We took a simple average because we confirmed each model to be individually well generalizable, so their predicted probabilities should not be very different (for most of the elements at least), so a simple average should suffice.

Thus our final output is thus created, and the process is completed.

4] Further Insights

While analysing the dataset, we made some useful observations and developed key insights, which we mention as follow as hypotheses with supporting evidence -

A) Transaction Triplets

There are a total of 664 columns whose names start with transaction_attribute, and as per the Problem Statement file, they have information on the number and value of transactions done using credit cards at various merchants.

We found that all 664 columns do not correspond to transactions done with different merchants. Instead every triplet of columns has information, with the middle column of the triplet having information on the number of goods purchased or similar quantity at the merchant. One example of a transaction triplet is the set of columns transaction_attribute_229, transaction_attribute_230 and transaction_attribute_231.

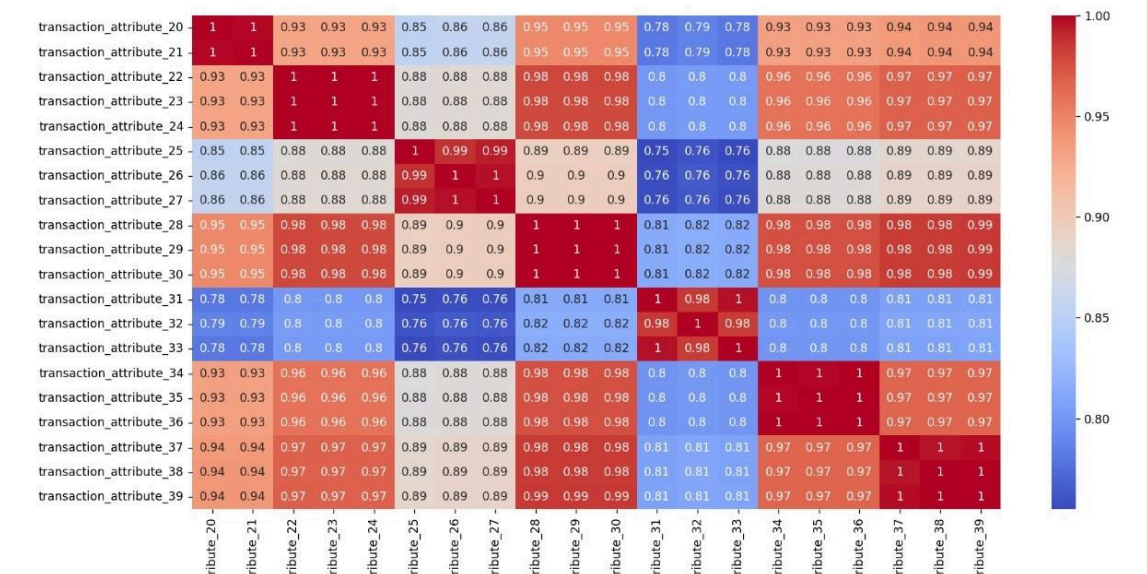
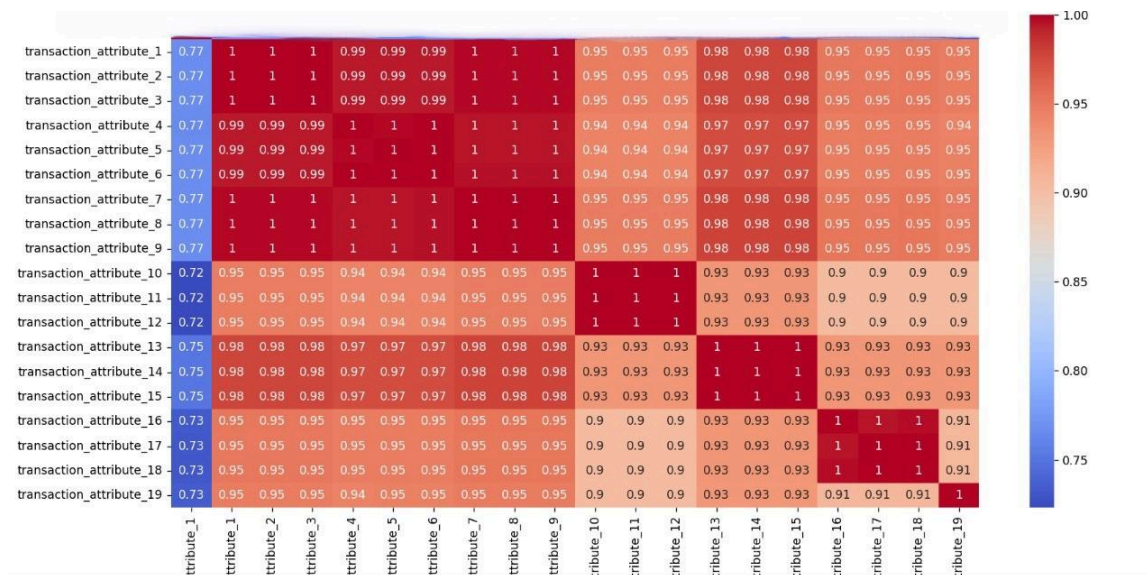
We arrived at this conclusion by analysing the **spearman correlation coefficient score** and **Mann Whitney U test p values** for a few columns.

As mentioned already, the Mann Whitney p value for a specific value is useful for measuring the way that specific column is affecting the target column. Similar p values suggest a similar effect on the target column, which can be due to similar distributions, and hence similar origins of the columns.

To confirm this we check the spearman correlation coefficient of the columns. This score is a statistical measure of the strength and direction of the association between two variables. It evaluates how well the relationship between two variables can be described using a monotonic function. A high score (close to +1 or -1) implies a strong relation between the columns, which combined with the p values suggest similar origins, which in our case imply from the same transaction

```
for col transaction_attribute_46 U-statistic: 38770799.0, p-value: 2.2080208665712618e-08
for col transaction_attribute_47 U-statistic: 38812415.5, p-value: 1.0735671724872334e-08
for col transaction_attribute_48 U-statistic: 38800135.0, p-value: 1.1381686851051268e-08
for col transaction_attribute_49 U-statistic: 38525512.0, p-value: 1.2936066640236145e-05
for col transaction_attribute_50 U-statistic: 38505001.5, p-value: 1.7370049713215096e-05
for col transaction_attribute_51 U-statistic: 38509049.0, p-value: 1.6567012178186962e-05
```

As can be seen above, the p values for the columns vary in a wide range, but for each triplet, the values are close. For the first triplet the values range from 0.44 - 0.51 and for the second triplet it ranges from 9.4×10^{-43} - 127.7×10^{-43} (although the range of scores seem to vary widely for the second triplet, it is important to remember the power of -43, which translates to a very small difference)



Both of the spearman correlation matrices provide further evidence for our hypothesis. As can be seen every column of a triplet has a very correlation with other members in it, while the same column's correlation with other columns are not as strong (they are still very strong, which we address in our second hypothesis)

Both of these images show the existence of triplets, and since the columns contain information on transactions with different merchants, we conclude that each triplet contains different information about the same transaction, or at least transactions with the same merchant

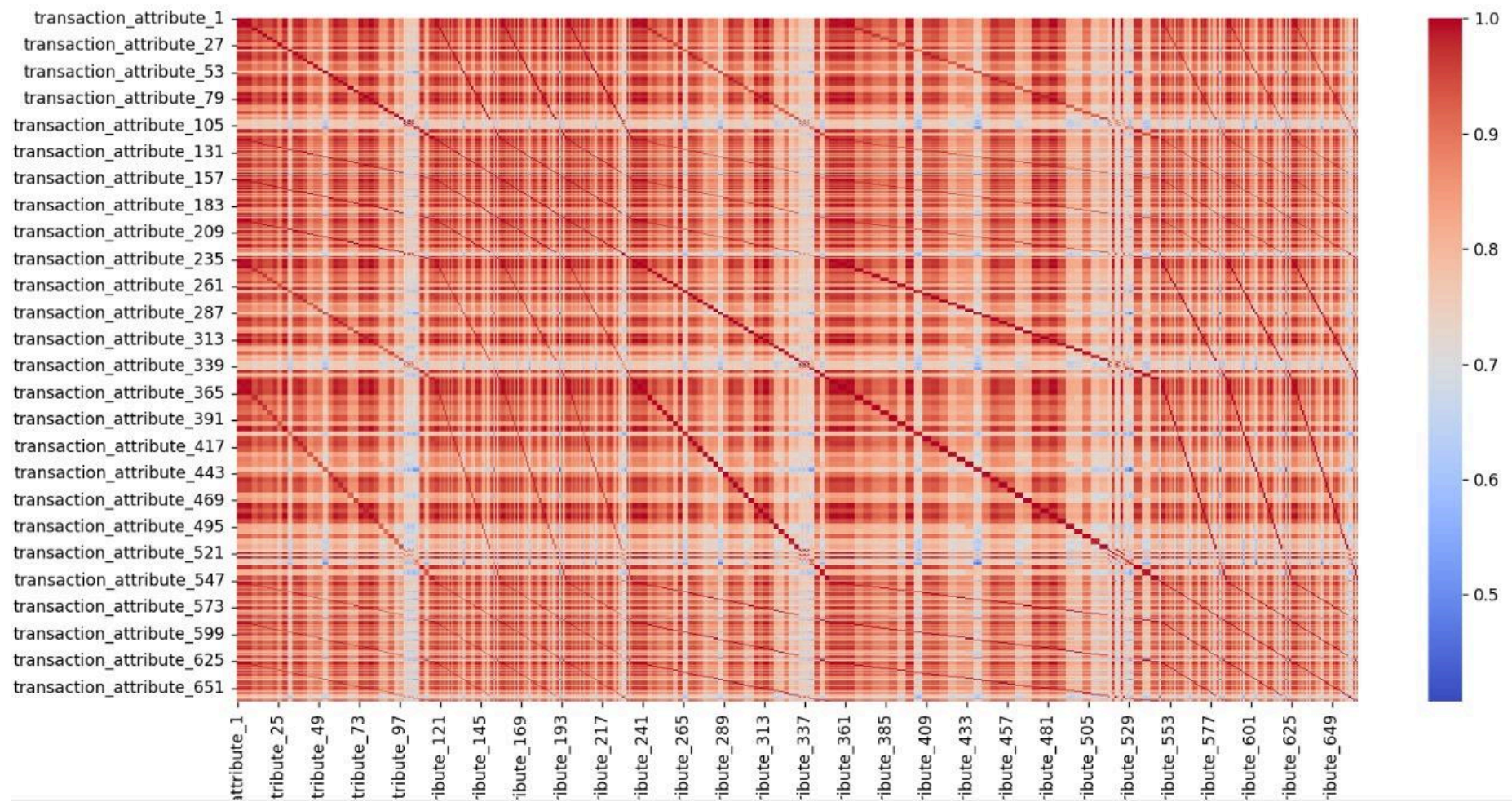
B) Behavioral Patterns in Credit Card Usage

Analysis of the transaction_attribute and onus_attribute columns indicate the presence of a very large group of customers with similar behaviour in the use of their credit cards, captured by simple trends in onus_attribute columns, which contain information about the credit card and related things like credit card limit, number of transactions per day limit etc.

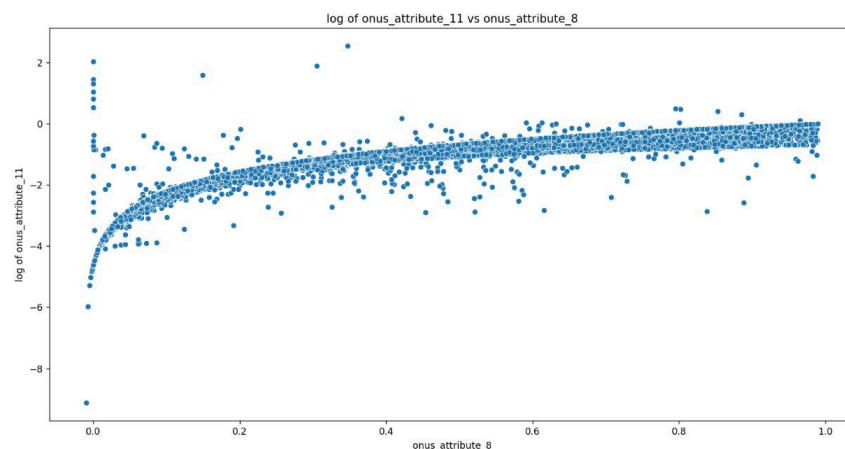
We arrive at this conclusion using the correlation matrices and plots between various columns. As we mentioned briefly above, most transaction triplets have a similar scoring and good relation with other triplets (means correlation scores between one member of first triplet and one member of second triplet are very close to other pairs, and most of such scores are high enough to indicate significant relation).

We further verify the high correlations between all triplets by analysing the 664 x 664 correlation matrix between all columns of transaction_attribute. **Only 108 of the 4,40,986** elements of this matrix have values less than 0.5, which is usually taken as a threshold to assess strong relations. Even among the 108 elements the **lowest value is 0.4**.

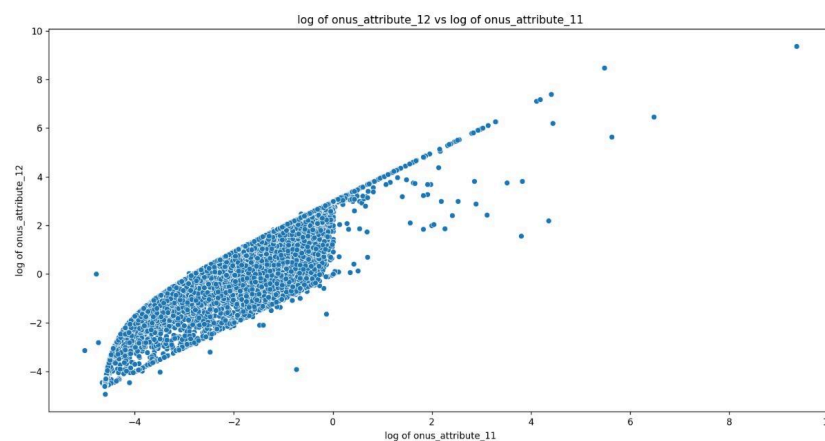
All of our findings mentioned above are based on this diagram shown below, which is the 664 x 664 spearman correlation matrix



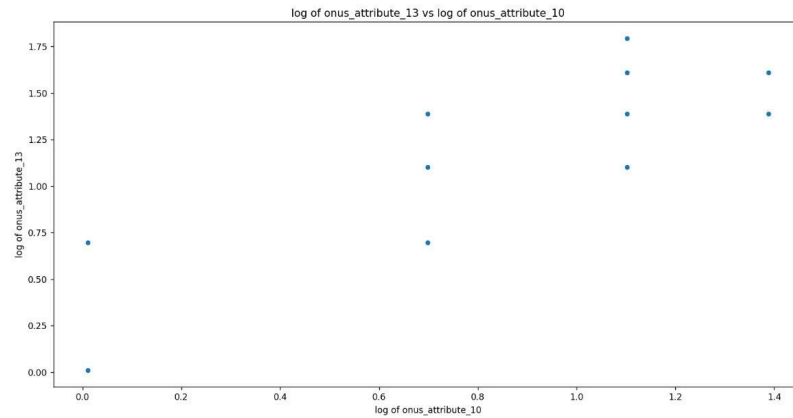
We also see strong correlations between various onus_attribute columns. We think of these columns as a group whose information comes from the same or similar origin. With this assumption, plots between some of the column combinations show simple analytic functions as the relation between the column values. These simple trends can be inferred from the fact that a majority of the customers follow a trend, which leads to smooth relation between these attributes. Since these attributes relate to properties of the credit cards, we infer that most of these customers are in a similar group with respect to the use of credit cards. The supporting plots are as follows -



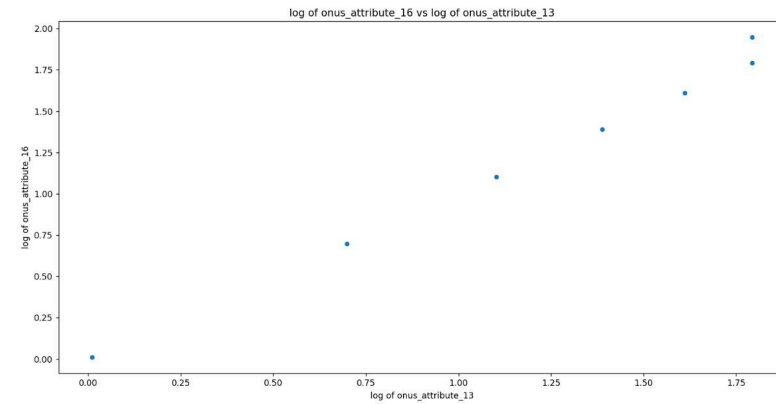
Log of onus_attribute_11 vs onus_attribute_8



Log of onus_attribute_12 vs log of onus_attribute_11



Log of onus_attribute_13 vs log of onus_attribute_10



Log of onus_attribute_16 vs onus_attribute_13

C) Temporal Consistency of Bureau Attribute Enquiries Across Time Periods

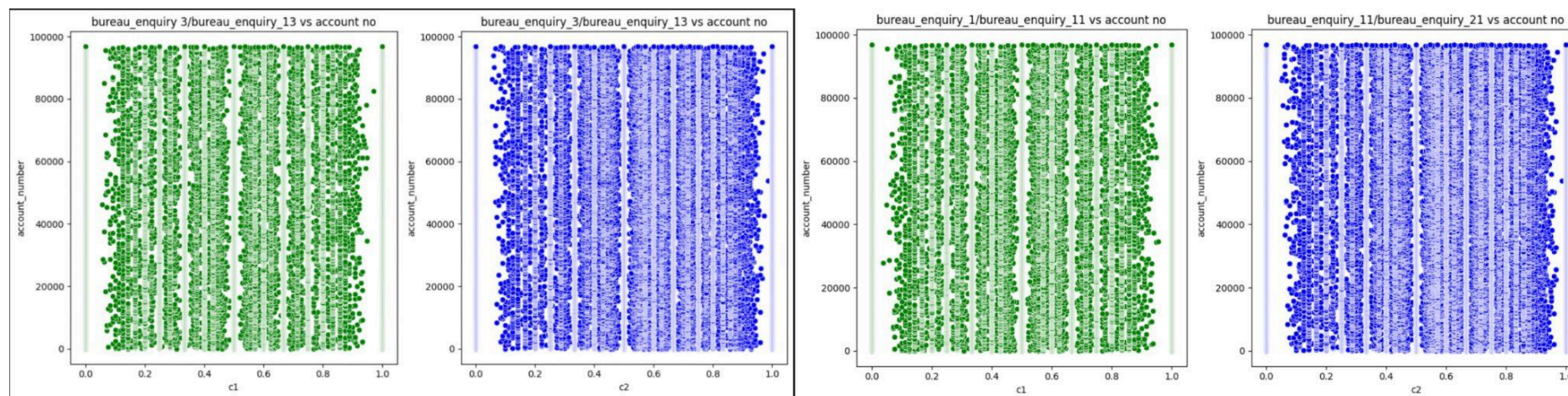
Analysis of the bureau_enquiry columns reveals consistent patterns of correlation, suggesting an underlying structure that organizes the data into distinct groups. According to data description these columns capture information related to bureau enquiry-level attributes, such as personal loan (PL) enquiries in the last three months and other related metrics.

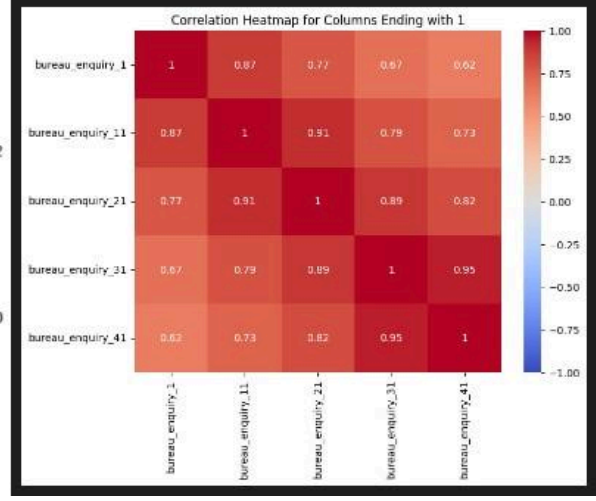
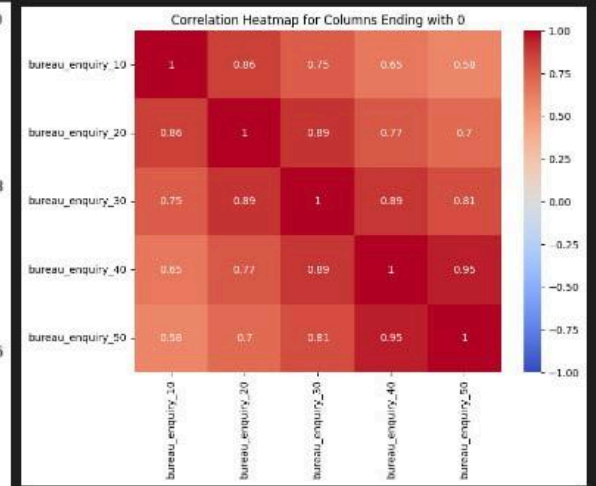
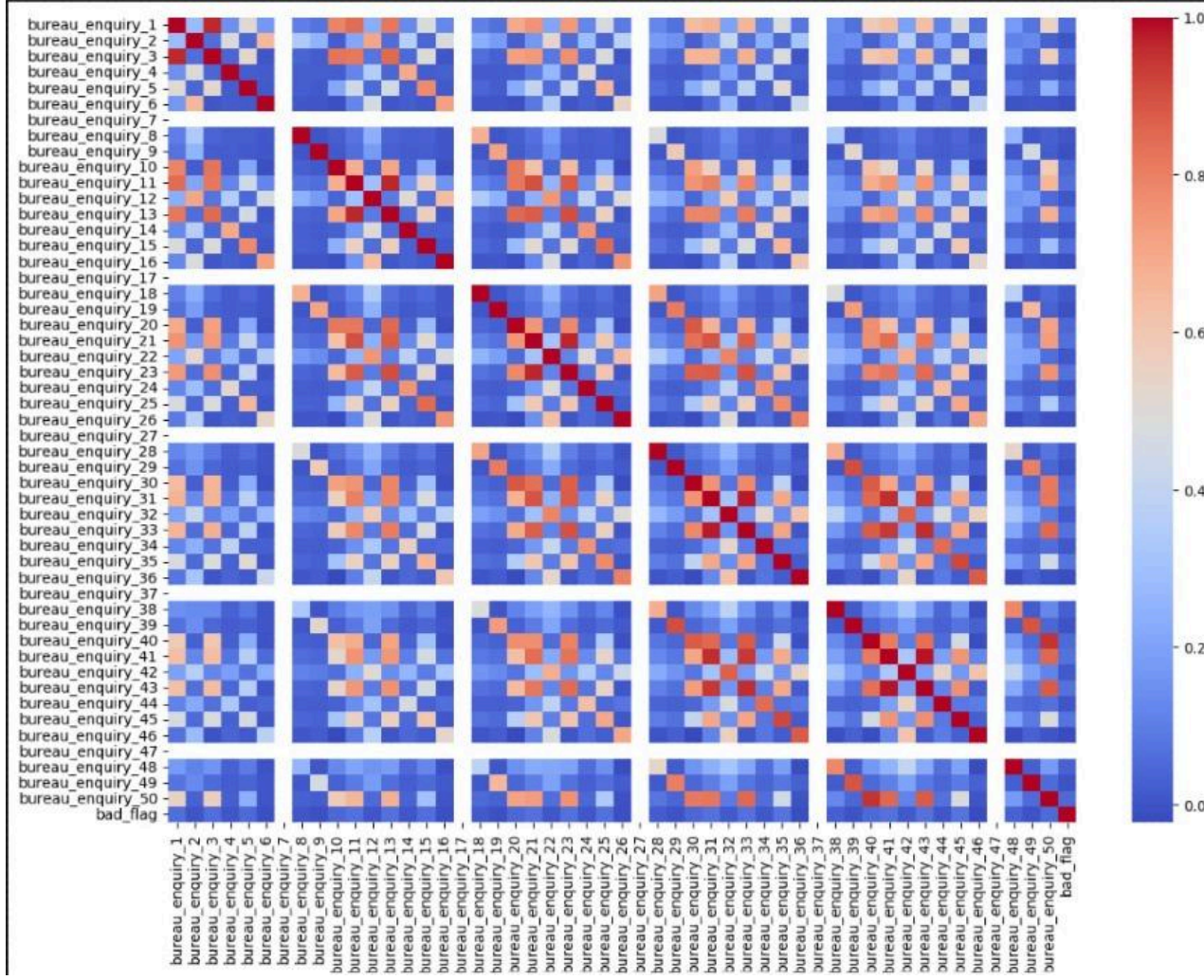
Using the heatmap of correlations, we observe strong relationships between columns with the same last digit (e.g., bureau_enquiry_1, bureau_enquiry_11, bureau_enquiry_21, etc.). These patterns imply that such columns are grouped based on a common enquiry type, possibly tracked over periodic intervals. We arrive at this hypothesis by analyzing the correlation matrix between the bureau columns.

We arrive at this conclusion by analyzing the correlation matrix between the bureau columns. The matrix highlights a consistent trend where columns sharing the same last digit exhibit high correlation coefficients. For example, the correlation scores between bureau_enquiry_1 and bureau_enquiry_11 are similar to those between bureau_enquiry_21 and

bureau_enquiry_31, indicating that they represent related enquiries across different periods. Additionally, columns ending with "7" appear to have minimal variation, often containing a single unique value, further supporting the structured nature of these features.

For further evidence we plotted a ratio of 2 columns vs account number for the same time interval like bureau_enquiry_1/bureau_enquiry_11 vs account number and bureau_enquiry_11/bureau_enquiry_21 vs account number and observed similar distribution. Same we did for columns ending with 3 ie. bureau_enquiry_3/bureau_enquiry_13 vs account number and bureau_enquiry_13/bureau_enquiry_23 vs account number.





5] Conclusion

Our report provides a detailed overview of the data analysis and preprocessing pipeline, the model design strategy, training and tuning methodology, and important insights into customer lifestyles and spending habits. Including this model in their risk management framework will help Bank A boost productivity and improve their standing in the market and economy.

Appendix A: Mann Whitney U Test scores

Due to space limitations, we show here only the scores of the top 20 columns for the first 3 subsets

Subset 1		Subset 2		Subset 3	
onus_attribute_2	6.051E-162	onus_attribute_26	5.3136E-162	onus_attribute_26	2.4538E-168
onus_attribute_26	1.2204E-153	onus_attribute_32	5.3026E-158	onus_attribute_2	1.8411E-164
onus_attribute_32	3.1732E-152	onus_attribute_2	3.1742E-156	onus_attribute_32	5.3915E-162
onus_attribute_17	2.6716E-150	onus_attribute_38	4.7682E-149	onus_attribute_17	3.015E-152
onus_attribute_29	7.3771E-144	onus_attribute_17	2.5654E-146	onus_attribute_38	6.6913E-149
onus_attribute_23	1.1921E-143	onus_attribute_23	1.0266E-142	onus_attribute_29	3.5093E-148
onus_attribute_20	1.3809E-143	onus_attribute_20	1.3467E-142	onus_attribute_23	6.1959E-146
onus_attribute_38	6.0702E-141	onus_attribute_33	2.3676E-142	onus_attribute_20	7.5161E-146

onus_attribute_35	1.8774E-136	onus_attribute_29	1.9576E-135	onus_attribute_33	6.8578E-142
onus_attribute_33	2.8639E-132	onus_attribute_39	7.3227E-135	onus_attribute_35	7.685E-140
onus_attribute_41	1.6189E-131	onus_attribute_35	1.0004E-131	onus_attribute_41	6.0721E-135
onus_attribute_39	4.0466E-124	onus_attribute_41	4.3141E-128	onus_attribute_39	5.275E-133
bureau_439	5.9025E-117	onus_attribute_27	3.686E-119	onus_attribute_27	9.0719E-125
bureau_452	1.36E-116	bureau_452	2.03E-118	bureau_439	5.1327E-117
bureau_450	1.5383E-114	bureau_450	4.4057E-118	bureau_452	1.2584E-116
onus_attribute_27	1.0273E-111	bureau_439	9.288E-118	bureau_450	2.597E-116
bureau_441	2.5948E-110	bureau_441	4.0012E-109	bureau_441	3.3272E-110
bureau_enquiry_45	1.6948E-101	bureau_enquiry_45	2.9183E-102	bureau_enquiry_45	1.0042E-99
bureau_enquiry_35	1.985E-101	bureau_enquiry_35	1.6154E-99	bureau_enquiry_35	7.77914E-97
bureau_enquiry_31	3.5574E-95	bureau_enquiry_41	1.42429E-93	bureau_enquiry_31	2.13201E-95

Appendix B: References

1] CATBOOST feature importance

<https://www.geeksforgeeks.org/catboost-feature-importance/>

2] Catboost

<https://www.geeksforgeeks.org/catboost-ml/>

https://catboost.ai/docs/en/concepts/python-reference_catboostclassifier

3] LIGHTGBM

<https://www.geeksforgeeks.org/lightgbm-light-gradient-boosting-machine/?ref=lb>

4] Mann Whitney U Test

<https://www.technologynetworks.com/informatics/articles/mann-whitney-u-test-assumptions-and-example-363425>

<https://statistics.laerd.com/spss-tutorials/mann-whitney-u-test-using-spss-statistics.php>

<https://docs.scipy.org/doc/scipy-1.15.0/reference/generated/scipy.stats.mannwhitneyu.html>

5] Train test split

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

<https://www.geeksforgeeks.org/how-to-do-train-test-split-using-sklearn-in-python/>

6] StandardScaler

<https://scikit-learn.org/1.6/modules/generated/sklearn.preprocessing.StandardScaler.html>

7] Filling NAN

<https://www.geeksforgeeks.org/how-to-fill-nan-values-with-mean-in-pandas/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html>

8] PCA

<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

<https://www.geeksforgeeks.org/implementing-pca-in-python-with-scikit-learn/>

9] Decision Tree

<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

<https://www.geeksforgeeks.org/decision-tree/>

<https://github.com/campusx-official/decision-trees>

10] XGBOOST

<https://www.geeksforgeeks.org/xgboost/>

<https://xgboost.readthedocs.io/en/stable/tutorials/model.html>

11] ROC AUC

<https://www.geeksforgeeks.org/auc-roc-curve/>

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html

12] Optuna

<https://docs.databricks.com/en/machine-learning/automl-hyperparam-tuning/optuna.html>

<https://optuna.readthedocs.io/en/stable/>

13] Stratified division

https://scikit-learn.org/stable/modules/cross_validation.html#stratified-k-fold

<https://www.scribbr.com/methodology/stratified-sampling/>

14] Bayesian Optimization

<https://github.com/bayesian-optimization/BayesianOptimization>

<https://towardsdatascience.com/bayesian-optimization-concept-explained-in-layman-terms-1d2bcdeaf12f>